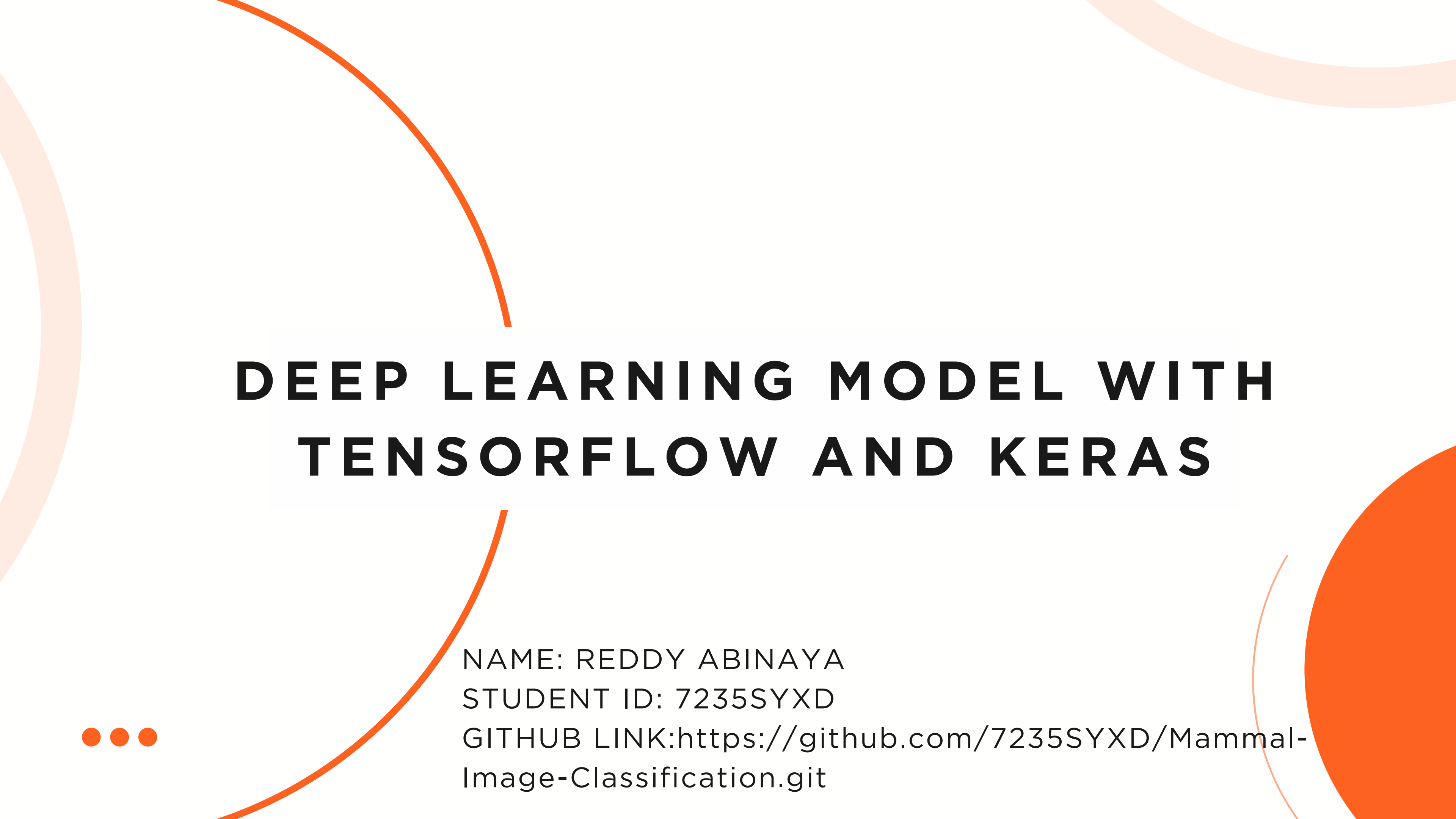




# DEEP LEARNING MODEL WITH TENSORFLOW AND KERAS

NAME: REDDY ABINAYA

STUDENT ID: 7235SYXD

GITHUB LINK:<https://github.com/7235SYXD/Mammal-Image-Classification.git>



# INTRODUCTION AND OBJECTIVES

1

## INTRODUCTION TO THE DATASET:

I load the Mammal Image Classification dataset from the Kaggle link provided <https://www.kaggle.com/datasets/asaniczka/mammals-image-classification-dataset-45-animals>.

2

## OBJECTIVE:

**Task:** Design, train, and evaluate deep learning image classification models using TensorFlow and Keras.

### Evaluation Metrics:

- Top 1 Accuracy - How often the correct species is predicted.
- Macro F1-Score - Treats all the 45 classes equally, regardless of image count.

4

5

## Why I Chose this dataset:

- There is a real data difficulty to discuss and resolve when there is a real class imbalance.
- Comparing visually similar animals, such as wolves and foxes, and leopards and cheetahs, makes the classification problem intricate and difficult.
- Real-world conditions are reflected in web-scraped photos of varying quality.
- The CNN vs. Transfer Learning comparison is significant and understandable due to the small per-class size (~306 images).
- Error interpretation does not require specialized subject expertise.

6

7

8

9

10

# PROBLEM DEFINITION AND DATASET UNDERSTANDING

1

2

3

4

5

6

7

8

9

10

## PROBLEM DEFINITION AND CHALLENGES

- Goal: Recognize 45 different mammal species from natural photos.
- Detailed recognition: Many species share similar shapes, textures, and patterns.
- Data issues: Varying backgrounds, lighting conditions, and image quality.
- Class imbalance: Different image counts per species.
- Visual similarity challenge.

## DATASET STATISTICS

- 45 Mammal classes
- 13,751 Total images
- 70/15/15 train/validation/test split
- ~306 Average images per class
- Stratified Split method

# EXPLORATORY DATA ANALYSIS

1

## CLASS IMBALANCE:

The bar graph shows the distribution that some classes have more images than the minority classes. High risk reveals there is an imbalance ratio. This is resolved using augmentation diversity and not by class\_weight, which destabilises gradient updates.

2

```
counts_series = pd.Series(class_counts).sort_values(ascending=False)
fig, ax = plt.subplots(figsize=(18, 5))
ax.bar(counts_series.index, counts_series.values,
       color='steelblue', edgecolor='none')
```

CODE SNIPPET 1: CLASS IMBALANCE

4

## RESOLUTION SPREAD

Native image sizes are varying from small to large photos. The images are resized to 128x128 for CNN modelling and 224x224 for MobileNetV2

5

```
width, height = PIL.Image.open(img_path).size
widths.append(width)
heights.append(height)
```

CODE SNIPPET 2: RESOLUTION SPREAD

6

7

## PIXEL STATISTICS

Every channel's mean and std is computed on training images. Red, green, and blue channels are normalized to [0,1]. Histogram confirms images have varying brightness which is satisfied by RandomBrightness augmentation

8

9

```
for channel_idx in range(3):
    channel_pixels[channel_idx].append(img_array[:, :, channel_idx].ravel())
except Exception:
    pass
```

CODE SNIPPET 3: PIXEL STATISTICS

10

# DATA PREPROCESSING AND AUGMENTATION

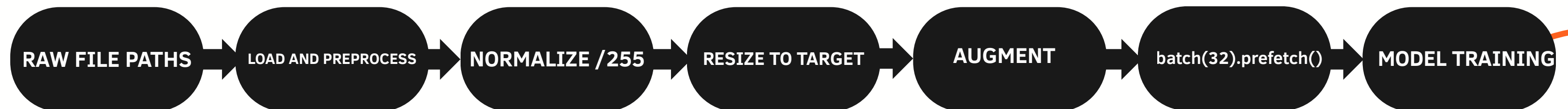
```
def load_and_preprocess(file_path, label):
    raw_bytes = tf.io.read_file(file_path)
    image = tf.image.decode_image(
        raw_bytes, channels=3, expand_animations=False)
    image = tf.cast(image, tf.float32) / 255.0
    image = tf.image.resize(image, img_size)
    return image, label

def apply_augment(image, label):
    image = augmentation_layer(image, training=True)
    image = tf.clip_by_value(image, 0.0, 1.0)
    return image, label
```

**CODE SNIPPET 4: AUGMENTATION LAYER(TRAINING ONLY)**

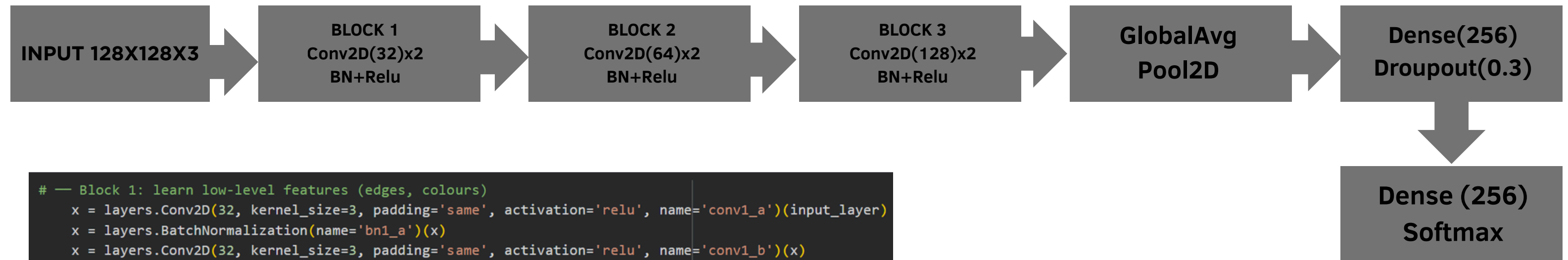
## DESIGN DECISION:

- Stratified split (70/15/15) - preserves all the images with class ratios.
- Augmentation is applied only to training data validation and testS are uncontaminated
- Class weights are not applied because it increases the loss % and decreases the train accuracy to ~2%.
- Image sizes vary for both the models 128x128 for CNN and 224x224 for Transfer learning
- prefetch()- CPU and GPU overlaps, so there is no I/O bottleneck





# MODEL A - CUSTOM CNN



```
# — Block 1: learn low-level features (edges, colours)
x = layers.Conv2D(32, kernel_size=3, padding='same', activation='relu', name='conv1_a')(input_layer)
x = layers.BatchNormalization(name='bn1_a')(x)
x = layers.Conv2D(32, kernel_size=3, padding='same', activation='relu', name='conv1_b')(x)
x = layers.BatchNormalization(name='bn1_b')(x)
x = layers.MaxPooling2D(pool_size=2, name='pool1')(x)
x = layers.Dropout(0.25, name='drop1')(x)

# — Block 2: learn mid-level features (textures, patterns)
x = layers.Conv2D(64, kernel_size=3, padding='same', activation='relu', name='conv2_a')(x)
x = layers.BatchNormalization(name='bn2_a')(x)
x = layers.Conv2D(64, kernel_size=3, padding='same', activation='relu', name='conv2_b')(x)
x = layers.BatchNormalization(name='bn2_b')(x)
x = layers.MaxPooling2D(pool_size=2, name='pool2')(x)
x = layers.Dropout(0.25, name='drop2')(x)

# — Block 3: learn high-level features (body parts, shapes)
x = layers.Conv2D(128, kernel_size=3, padding='same', activation='relu', name='conv3_a')(x)
x = layers.BatchNormalization(name='bn3_a')(x)
x = layers.Conv2D(128, kernel_size=3, padding='same', activation='relu', name='conv3_b')(x)
x = layers.BatchNormalization(name='bn3_b')(x)
x = layers.MaxPooling2D(pool_size=2, name='pool3')(x)
x = layers.Dropout(dropout_rate, name='drop3')(x)
```

## CODE SNIPPET 5: CUSTOM CNN

### DESIGN RATIONALE:

- There are 3 filters, 32->64->128, which learns edges, textures, and shapes
- BatchNorm is used to stabilize the gradient flow and does not require L2.
- GlobalAvgPool stabilises the params, which in turn improves invariance.
- Dropout 0.25-0.3 helps to prevent memorisation.
- 128x128 pixel size input helps 3x faster for scratch training rather than 224.
- Class weights are not applied because it causes accuracy to fall drastically, which is ~2%.

# MODEL B- MobileNetV2 Transfer Learning

## STAGE 1 - HEAD ONLY (BASE FROZEN):

- The MobileNetV2 base is not trained and is kept frozen because of some similarity in certain species images.
- Dense heads are trained
- Adam optimizer is used due to which  $lr=1e-3$ .
- Establishes class mapping from ImageNet features.
- It is converged at the best epoch, and the best epoch is 19.

## STAGE 2 - FINE-TUNE TOP 30 LAYERS:

- First load the best weights, which are the most critical part.
- Unfreeze the top 30 of 154 base layers.
- To prevent catastrophic forgetting  $lr = 5e-5$ .
- Low-level features are kept in lower layers for ImageNet.

```
#Stage 1: Base frozen
tl_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=lr_tl_head),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'],
)
# Stage 2: Load weights-> unfreeze top 20-> retains at tiny LR
tl_model.load_weights('best_tl_stage1.keras')
best_stage1_val_acc = max(history_tl_stage1.history['val_accuracy'])
print(f'Loaded best Stage 1 weights (val_accuracy = {best_stage1_val_acc:.4f})')
unfreeze_top_layers([base_model, num_layers_to_unfreeze=30])
tl_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=lr_tl_fine),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'],
)
```

CODE SNIPPET 6: FOR HEAD ONLY AND FINE TUNNING

# TRAINING SETUP AND CALLBACKS

## MODEL CHECKPOINT:

It saves only the best val\_accuracy epoch and prevents final epoch weights from being degraded.

## EARLY STOPPING:

When validation accuracy stops increasing for 10-15 epochs, it results in restoring the best weights automatically.

## ReduceLROnPlateau:

Allows fine-grained steps near the minimum, which halves the learning rate when val\_loss plateaus for 4-5 epochs.

## TENSORBOARD:

Enables interactive curve inspection after training on log metrics per epoch.

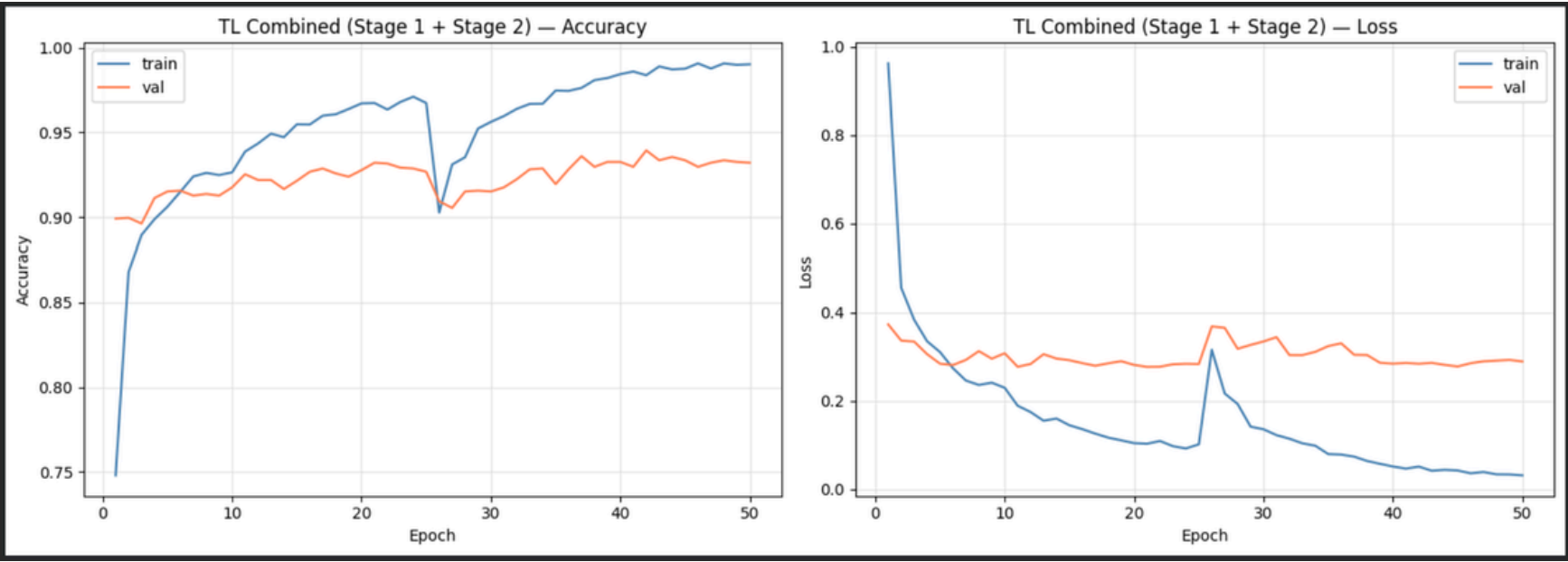


FIGURE 1: COMBINED ACCURACY AND LOSS FOR BOTH THE STAGES

| MODEL      | BEST EPOCH  | BEST VAL ACC | LR STRATEGY    | STOPPED AT    |
|------------|-------------|--------------|----------------|---------------|
| CUSTOM CNN | EPOCH 18/60 | 36.02%       | 3e-4 → 9.4e-6  | EPOCH 33      |
| TL STAGE 1 | EPOCH 19/25 | 92.63%       | 1e-3 → 5e-4    | ALL 25 EPOCHS |
| TL STAGE 2 | EPOCH 21/25 | 93.6%        | 5e-5 → 1.25e-5 | ALL 25 EPOCHS |



# MODEL EVALUATION

## EVALUATION RESULTS - TEST SET

- **CNN Val Accuracy- 36.02%**
- **TL Stage 2 val Accuracy- 93.60%**
- **TL Error Rate- 6.3%**
- **TL Train-Val Gap- 5.4%**

| METRIC          | CUSTOM CNN | MobileNetV2 TL | IMPROVEMENT |
|-----------------|------------|----------------|-------------|
| TOP 1 ACCURACY  | 35.97%     | 93.70%         | +57.73 pp   |
| MACRO F1-SCORE  | ~0.34      | ~0.94          | +0.60       |
| MACRO PRECISION | ~0.43      | ~0.94          | +0.51       |
| MACRO RECALL    | ~0.35      | ~0.93          | +0.58       |
| TRAIN-VAL GAP   | 21.61%     | 4.6%           | -17.01 pp   |

FULL METRIC COMPARISION - TEST SET

# ERROR ANALYSIS

## CUSTOM CNN - OVERFITTING ANALYSIS

Train accuracy - 57.62% at best epoch 33

Val accuracy - 36.02% at best epoch 18

Gap - 21.60% - mild overfitting

### Reason for overfitting:

Even with approximately 306 images are not enough for training a scratch CNN without memorisation.

### Preventive measures:

- Dropout (0.25-0.3)
- BatchNormalization
- Data Augmentation
- EarlyStopping
- Reduced LR

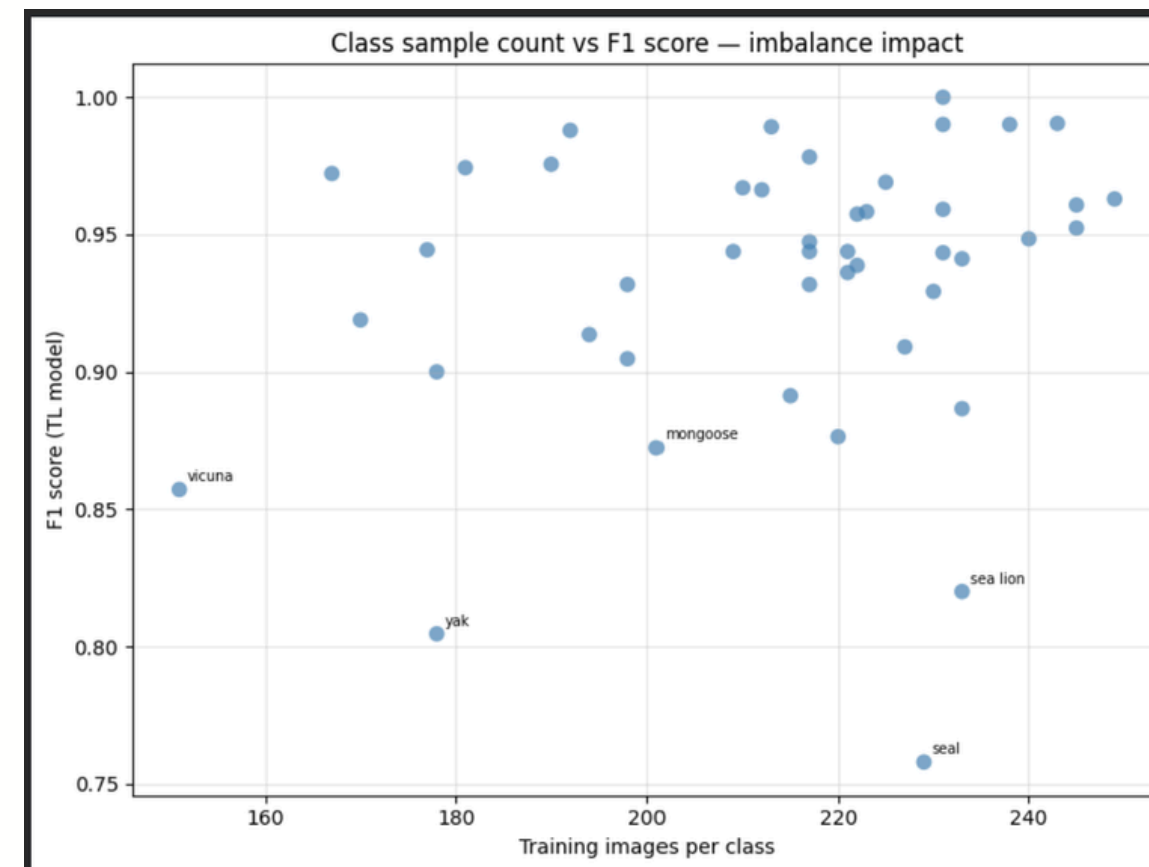


FIGURE 2: IMBALANCE IMPACT

## TL MODEL - ERROR ANALYSIS

Error rate - 6.3%

Train Val gap - 4.6% (Good Fit)

### Error Patterns:

- Similar species images are more confusing.
- Fewer train images, i.e, minority classes shows a lower per-class F1 score.
- There is an imbalance in the primary error driver between sample count and F1.
- Species with similar habitats are more confusing because there is the presence of background noise.

### Improvement:

- Focal loss
- Mixup augmentation
- EfficientNetB3
- Grad-CAM

1

2

3

4

5

6

7

8

9

10



## CONCLUSION AND SUGGESTION

- From all the analysis done, we encourage to use transfer learning model over scratch models, as we can see MobileNetV2 achieved 93.6% validation accuracy compared to a scratch CNN, which only achieved 36.02%.
- Because they continuously received lower F1 scores in the evaluation, concentrate on interventions for minority species classes that have previously performed badly due to a lack of training images.
- Since the CNN with 1.2 million parameters overfitted badly with just 306 images per class and a 21.61% train-val gap, encourage a good balance between model complexity and available data.
- To further reduce the performance difference for underrepresented classes, more study on sophisticated regularization strategies like Mixup augmentation and Focal loss could be required.

1

2

3

4

5

6

7

8

9

10



***THANK  
YOU***